CS405 Project 1 | Draft report

Efe Çağan Akkuş · Student ID: **32413**

Transformations, Projection and Curves from Paintings
Fall 2026 · Revised 12 November 2026 · Code baseline: e0677d5 + measurement JSON viewer

Painting and scene

Kavunlu Natürmort, Şeker Ahmed Paşa (Sakıp Sabancı Museum collection) was selected for the melon slice's clear organic outline and of course melon is a precious fruit that consumed in summer season with cheese being my favorite fruit. That's why melon painting is chosen. The painting is a textured quad. Three foreground solids demonstrate transforms; a table with a rotating platform demonstrates hierarchy; curves trace the lower edge of the melon slice. The painting image is taken directly from the supplied Week 01 asset [2]; its JPEG bytes are unchanged. This report follows the handout [1].

One Vite application uses raw WebGPU and WGSL. Mathematics, scene traversal, rendering, interaction and UI are separate modules. The common pipeline is $\mathbf{p}_{clip} = \mathbf{P}\,\mathbf{V}\,\mathbf{M}\,\mathbf{p}_{object}$. Four views share the application, reset controls and debug switch. Device/context, buffers, depth and render-loop structure reference Lab 3 [3]; basic WebGPU setup also appears in Lab 1 [12].

A. Transformations and order

Matrices use column-major storage and act on column vectors; the rightmost transform acts first. The matrix module implements identity, multiplication, translation, scaling, rotations around X/Y/Z, transpose and rigid inverse. For a rigid matrix [R,t;0,1], orthogonality gives the inverse $[R^T,-R^Tt;0,1]$. This shortcut is not valid for a scaled transform. Matrix multiplication, translation, scaling and rotations use the standard column-major formulas also provided in Lab 3 [4]; the rigid-inverse formula is covered in [5,6]. These are formula correspondences, not a claim that the complete library was copied.

The middle object has XYZ translation, Y rotation and uniform scale controls. Its rotation also includes a fixed -20° X tilt: R = $R_y(32°)\,R_x(-20°)$ in the captures. With T R S, local geometry is scaled, rotated and then translated. With S R T, the y_x translation vector itself is rotated and scaled, changing the position even though the controls are identical. The rest of the scene uses parent-to-child composition and local T R S ordering where applicable.



Figure 1. TRS (left) and SRT (right) screenshots with the scene and both on-screen matrix panels visible. Controls are identical. The numerical matrices are also reproduced below for readability; full captures retain the UI.

```
TRS (rows)                          SRT (rows)
[ 0.339 -0.072  0.199  0.280 ]      [ 0.339 -0.072  0.199  0.312 ]
[ 0.000  0.376  0.137 -0.520 ]      [ 0.000  0.376  0.137 -0.072 ]
[-0.212 -0.116  0.319  0.900 ]      [-0.212 -0.116  0.319  0.288 ]
[ 0.000  0.000  0.000  1.000 ]      [ 0.000  0.000  0.000  1.000 ] Controls:
t=(0.28,-0.52,0.90), scale=0.40, Y angle=32 deg.
```

The translation column directly verifies the origin's image because multiplying by (0,0,0,1) selects that column. A direction has homogeneous w = 0, so translation does not change it.

B. Scene graph and a leaf world matrix

Each node stores its local transform, parent, children and world matrix. Traversal computes $W_{child} = W_{parent} L_{child}$. The parent-first traversal follows the course hierarchy pattern [5], also described in [3]. There are nine nodes across four levels, including the root. Rotating the turntable carries both blocks without separately editing their world positions.

```
Scene
  Table
    Front left leg      Front right leg
    Back left leg       Back right leg
    Turntable
      Small block
      Tall block
```

The table and blocks have separate mesh-sizing matrices. These affect the geometry but do not scale the frame inherited by a child. The rendered model is $W_{node} S_{mesh}$; local debug axes use W_{node}. This keeps a thin tabletop from shrinking its descendants. Cube faces use the same conventional two-triangle index pattern as the lab [3], with face colors replacing its normal attributes.

Numerical verification at the reset pose

For the small block, angles are in radians and the animation is paused at the reset angle 0.28. The complete chain of local frames is:

```
W = T(0,-0.28,1.03) Rx(0.32) T(0,-0.24,0)
    T(0,0.105,0) Ry(0.28) T(-0.27,0.13,0)

W_small =
  [  0.96106  0.00000  0.27636 -0.25948 ]
  [  0.08693  0.94924 -0.30232 -0.30822 ]
  [ -0.26233  0.31457  0.91227  1.09926 ]
  [  0.00000  0.00000  0.00000  1.00000 ]
```

By hand, the block origin first becomes (-0.27,0.13,0). The turntable rotation sends it to (-0.27 cos(0.28),0.13,0.27 sin(0.28)). The table and turntable translations leave y = -0.005 before the root rotation. Applying R (0.32) and the root x translation yields **(-0.25948, -0.30822, 1.09926)**, matching the last column above to rounding. Mesh scale diag(0.19,0.16,0.19,1) is applied only after this frame is obtained.

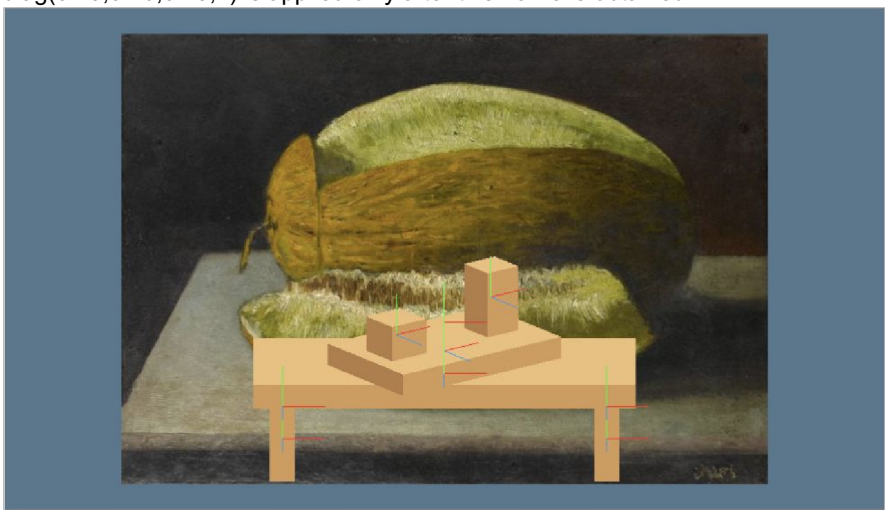


Figure 2. Scene detail with local axes. The paused capture is later than the reset pose used in the calculation above; the tree is reproduced above and in the full capture.

Reproducible verification - 11 November

All 13 Node tests passed with npm test. They cover rigid inverse, homogeneous directions, rotation order, camera degeneracy, projection/frustum corners, the leaf origin and hierarchy reset, plus curve interpolation and editing. The rotation oracle is adapted from V3.rot [9]. The production build also passes; these numerical checks do not replace browser interaction tests.

C. View and projection

Deriving lookAt

For eye e, target a and reference up h, define the backward unit axis w = normalize(e - a), the right axis u = normalize(h × w), and v = w × u. Taking coordinates relative to e gives $x_{view} = u \cdot (p - e)$, and similarly for v and w. This is the lookAt view derivation in the viewing lecture [6] and lab implementation [4]. The rows are:

```
V = [ ux uy uz -dot(u,e) ]
    [ vx vy vz -dot(v,e) ]
    [ wx wy wz -dot(w,e) ]
```

```
[  0   0   0     1     ]
```

If eye and target coincide, the implementation rejects the camera. If up is parallel to w, their cross product cannot define u; an alternate reference axis is selected before normalization. Mouse orbit updates eye around the target. The initial eye is approximately (0.45,0.61,5.04), target (0,-0.12,0.52). Our fallback-axis choice and coincident-eye rejection differ from [4].

WebGPU depth convention

View space looks down -Z. Let f = 1/tan(FOV /2), aspect r, near n and far F. The matrices below are written as rows, y although stored column-major. The perspective formula matches [4], with the projection/depth convention discussed in [6]:

```
Perspective:                  Orthographic (half-height h):
[ f/r  0    0          0 ]     [1/(hr)  0     0        0 ]
[  0   f    0          0 ]     [  0    1/h    0        0 ]
[  0   0  F/(n-F)  Fn/(n-F)]   [  0     0  1/(n-F) n/(n-F)]
[  0   0   -1          0 ]     [  0     0     0        1 ]
```

At z = -n, perspective division gives z = 0; at z = -F it gives 1. Orthographic has w = 1 and the same endpoints.

NDC

Perspective w = -z causes foreshortening; orthographic size is independent of distance. With n = 0.65 and F = 6.5, F/n = 10. Keeping near away from zero improves perspective depth precision while preserving the scene; reducing far can further limit the covered range. Precision remains nonuniform in perspective.

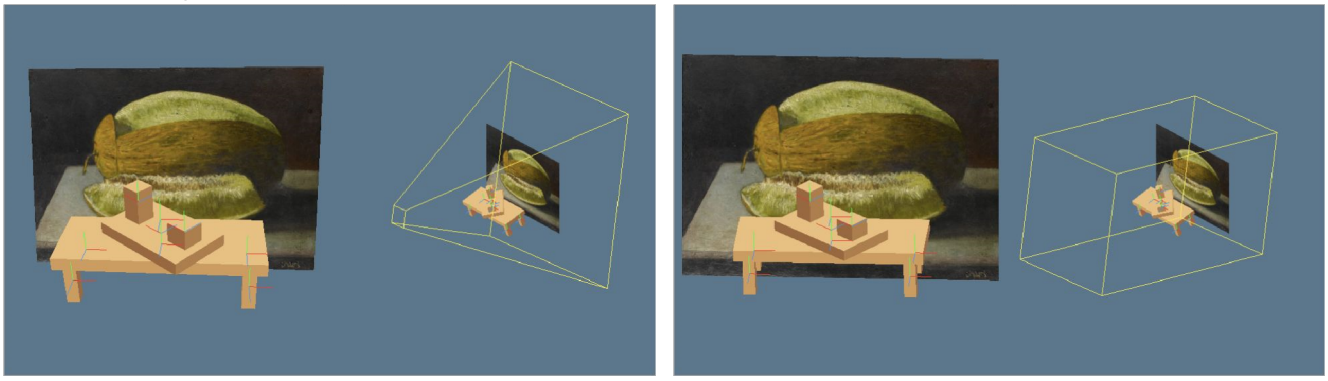


Figure 3. Canvas details: perspective (left), orthographic (right), at the same eye, target and frozen object pose. Each includes the inspection camera and yellow frustum; turntable animation is paused across the pair.

The eight near/far plane corners are transformed back to world space with V inverse and connected by twelve edges. The fixed inspection camera projects those world-space lines, exposing a perspective frustum versus an orthographic box.

Texture convention - checked against Week 07

The quad uses top-left UV (0,0), an rgba8unorm-srgb color texture, linear min/mag filtering and clamp-to-edge. Sampling decodes to linear RGB; the sRGB attachment encodes the output. These settings were compared with the Week 07 texturing example [10]; the texture code predates that comparison and was not copied from the new deck. Only the base level exists: mipmapping and anisotropic filtering are not implemented, so minification aliasing remains possible.

D. Curves traced over the melon

Cubic basis, endpoints and de Casteljau

For a segment with control points P through P , the Bernstein weights are $(1-t)^3$, $3(1-t)^2t$, $3(1-t)t^2$ and t^3. The curve is their
0 3
weighted sum [8, slides 11 and 13]; the same cubic formula appears in the original project demo [11]:

```
B(t) = (1-t)^3 P0 + 3(1-t)^2 t P1
     + 3(1-t)t^2 P2 + t^3 P3,     0 <= t <= 1
B'(t) = 3(1-t)^2(P1-P0) + 6(1-t)t(P2-P1)
      + 3t^2(P3-P2)
```

At t = 0 the weights become (1,0,0,0), so B(0) = P ; at t = 1 they become (0,0,0,1), so B(1) = P . De Casteljau repeatedly

0 3
interpolates adjacent points $Q_i = (1-t)P_i + tP_{i+1}$, reducing four points to three, then two, then one. The final point equals B(t); the intermediate levels are drawn for the active segment. The lerp/level construction and derivative follow the same mathematics as C6.levels and C6.dbez [7], implemented here with cubic weights and array operations.

Three segments and C1 continuity

Ten control points define segments [P0..P3], [P3..P6] and [P6..P9]. Adjacent curves share an endpoint (C0). For equal segment durations, C1 requires equal endpoint derivatives: 3(P3-P2) = 3(P4-P3), hence **P4 = 2P3-P2**; likewise P7 = 2P6-P5. Initial joins satisfy P3-P2 = P4-P3 = (0.18,-0.01) and P6-P5 = P7-P6 = (0.05,0.10). The mirrored-handle rule appears in [7, enforce], [8, slide 16] and the original demo [11, enforceC1].

Dragging an interior handle mirrors its partner about the shared anchor; dragging an anchor also moves its neighboring handles. Moves that would put a point or its mirrored handle outside [0,1] are rejected. The global parameter assigns equal time to each segment. Animation uses the derivative direction for the red arrow; uniform parameter steps do not imply constant arc-length speed [8, slides 14-15]. The painting-plane arrow is our geometry implementation of that tangent-orientation principle.

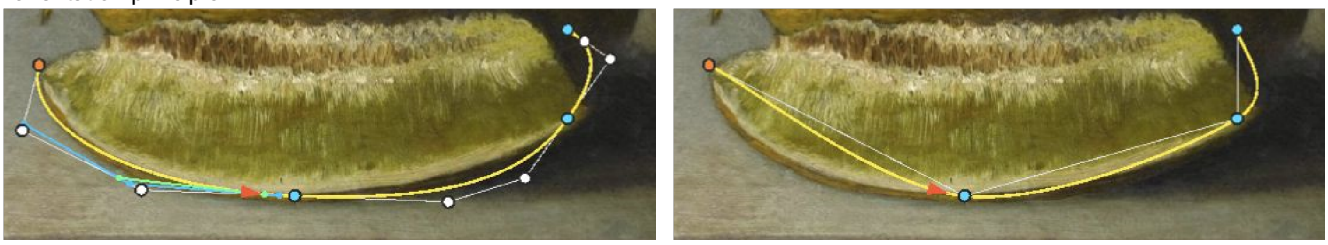


Figure 4. Enlarged melon-contour details: C1 Bézier and de Casteljau (left), Catmull-Rom (right). Global t = 0.300; the Bézier segment-local t is 0.900. Full painting captures are retained in docs/screenshots/.

Second family and export

The uniform Catmull-Rom segment interpolates the middle pair of four knots. The revised implementation directly adapts C6.crBez [7]: convert the central-difference tangents to cubic Bézier handles, then reuse the existing evaluator and derivative.

```
B0 = P1                    B1 = P1 + (P2-P0)/6
B2 = P2 - (P3-P1)/6         B3 = P2
```

Endpoint knots are repeated; the four Bézier anchors become spline knots. A 51-parameter comparison against the independent expanded polynomial passes, as do finite-difference tangent checks. The lecture examples also pass: the cubic gives (4,3) at t=0.5 [8, slide 9]; Catmull-Rom gives (3,1) [8, slide 20]. Those exact worked examples are used as test data. Both C1 joins remain valid after anchor/handle edits. Export uses bottom-left normalized coordinates; Appendix A reproduces exports/curve.json.

E. Integration, measurements and appendices

One application provides A-D views, reset/debug and recorded-JSON controls. Run npm install then npm run dev, or npm run build and npm run preview. All seven figures were refreshed on 11 November; full captures are in docs/screenshots/.

Measured performance

Recorded 11 November on the current production build. Apple M2 (10 GPU cores), macOS 26.6.2; Codex in-app browser, reported Chromium UA 156.0.0.0. Fixed viewport 1280 × 900 CSS pixels; canvas 980 × 558, DPR 1. Each run uses 3 s warm-up and at least 30 s foreground sampling, retaining all intervals. Animation/debug are on for B/C/D; de Casteljau is Bézier-only. Hiding/resizing cancels recording. Build fingerprints: docs/measurements/build.json.

Scene	Mean ms	P95 ms	Max ms	FPS	CPU ms	Vertices
A	16.67	17.20	17.70	60.00	0.21	114
B	16.67	17.30	17.70	60.00	0.25	348
C	16.67	17.10	17.70	60.00	0.24	720
D-B	16.67	18.30	18.80	60.00	0.87	2169
D-C	16.67	18.60	18.80	60.00	0.74	1467

D-B: Bézier; D-C: Catmull-Rom. FPS = 1000/mean interval; P95: nearest rank. CPU time ends at queue.submit; neither metric measures GPU execution. Near-60 FPS pacing and different debug geometry prevent isolating formula cost. One run per mode gives no variance estimate. Raw samples: docs/measurements/.

Debugging and practical limitations

The preview server had to be restarted. Inconsistent downloads led to the Recorded JSON viewer; all runs were repeated at a fixed resolution on that build. Camera captures freeze the same object pose. Old measurements were removed. No GPU timestamps or cross-device tests are claimed.

Appendix A. Exported curve (compact JSON)

{"painting":"Kavunlu Natürmort","artist":"Şeker Ahmed Paşa",

"type":"piecewise-cubic-bezier",

"coordinates":{"units":"normalized","origin":"bottom-left",

"xRange":[0,1],"yRange":[0,1]},

"segments":3,"continuity":"C1","controlPoints":[

{"x":0.18,"y":0.40},{"x":0.16,"y":0.29},{"x":0.30,"y":0.19},

{"x":0.48,"y":0.18},{"x":0.66,"y":0.17},{"x":0.75,"y":0.21},

{"x":0.80,"y":0.31},{"x":0.85,"y":0.41},{"x":0.82,"y":0.44}, {"x":0.80,"y":0.46}]}

Appendix B. AI usage

OpenAI Codex assisted with planning, debugging, tests, measurement tooling and evidence. It also supported report drafting and final editing for grammar, clarity and academic tone.

References - instructor-provided CS405 materials, Fall 2026

[1] CS405_Project1_2026.docx. Assignment requirements and evidence specification.
[2] Week01/assets/kavunlu.js. Painting image reused as public/assets/kavunlu.jpg.
[3] Week04/lab3-starter/main.js. WebGPU structure reference; cube packing comparison.
[4] Week04/lab3-starter/mat4.js. Standard matrix, lookAt and perspective formula comparison.
[5] Week02./Week02/S3-Thu-Oct08-Transformations-SceneGraphs-IK.html. Node.update traversal and rigid inverse.
[6] Week04/L3-Thu-Oct22-Viewing-Projection-Pipeline.html. View basis, projection and depth precision.
[7] Week06/demos/w6a.js. C6.crBez directly adapted; lerp/levels/dbez/enforce formulas compared.
[8] Week06/L1-Mon-Nov02-Curves.html. Slides 9/20 test data; 11/13 basis/properties; 14-16 motion/C1.
[9] Week06/demos/w6b.js, V3.rot. Yaw-then-pitch formula adapted into the matrix test only.
[10] Week07/L1-Mon-Nov09-Texturing.html, slide 22 / Level 10. Later texture/sampler validation only.
[11] Project1.zip, Project1/p1.js. Original curve/tracing demo: bez, crPoint, curveAt, enforceC1.
[12] Week02./Week02/lab1-solution/main.js. Matching basic WebGPU device/context API reference.

Detailed project-function mapping: docs/code-attribution.md.